

**INTERNATIONAL CYBERSECURITY
AND DIGITAL FORENSICS ACADEMY**

**INTRO TO CRITICAL INFRASTRUCTURE
FOR IT LEARNERS**

LAB 02 — PLC PROGRAMMING LANGUAGES

Student Edition

Authorized ICDFA training environments only
icdfa.edu.ng

Lab 02: PLC Programming Languages

Course: Intro to Critical Infrastructure for IT Learners | **Estimated Duration:** 2–3 hours

Submission deadline: As published in ICDFA E-Campus.

Authorization & Safety: Perform all activities only on the ICDFA-provided virtual machines, packet captures and isolated training networks. Do not bridge these VMs to production networks, scan external systems, or reuse credentials observed in training data.

Learning Outcomes

- Compare the PLC programming languages presented in the course.
- Interpret a motor start/stop latching circuit in Ladder Diagram.
- Recognize equivalent motor-control logic represented using Function Block Diagram, Structured Text, Instruction List and Sequential Function Chart.
- Explain how tags and Boolean data types support PLC logic.

Laboratory Environment

System	Expected Address	Account	Password
PLC VM	192.168.1.10	plc	password

Note: If the instructor-provided VM image uses updated addressing or credentials, use the values published in E-Campus. Do not alter addressing unless instructed.

Pre-Lab Checklist

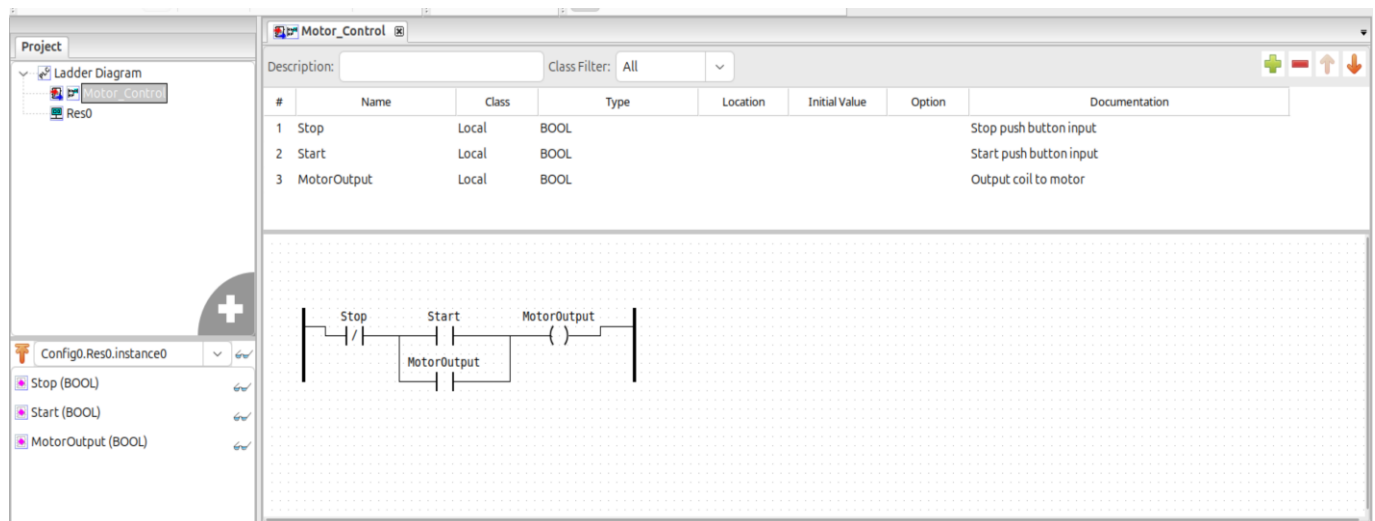
- Read the complete lab before beginning.
- Confirm you can access the required VM(s) and that virtualization support is enabled.
- Use the instructor-specified isolated/host-only network.
- Create or restore the clean starting snapshot for this lab.
- Prepare a report document with your name, registration number, course, lab number and date.
- Capture evidence as you progress rather than repeating the lab at the end.

Procedure

Step 1. Start from the clean ICDFA PLC VM snapshot and sign in to the PLC training environment.

Step 2. Launch OpenPLC Editor and open the Ladder Diagram example from the PLC Programming Language Examples directory.

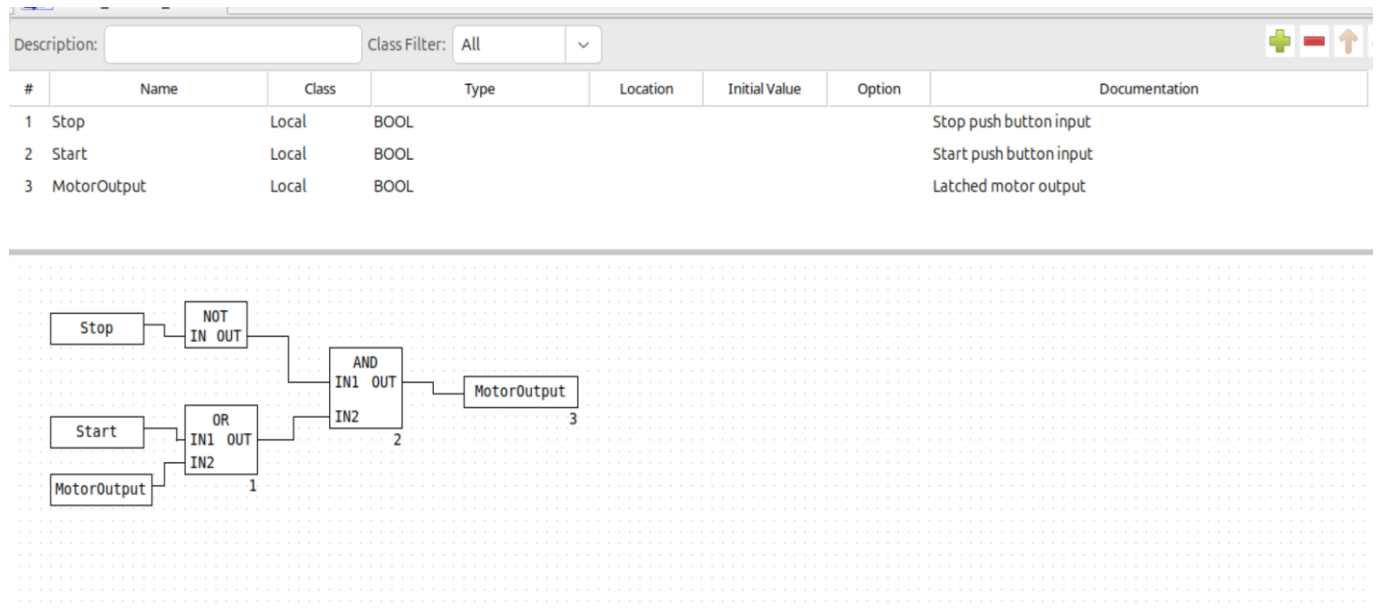
Step 3. Open Res0 > instance0 (Motor_Control). Identify the normally open Start contact, normally closed Stop contact, MotorOutput latch contact and MotorOutput coil.



Step 4. Open the tag/variable list. Confirm that the motor-control tags use Boolean values and record the purpose of each tag in your worksheet.

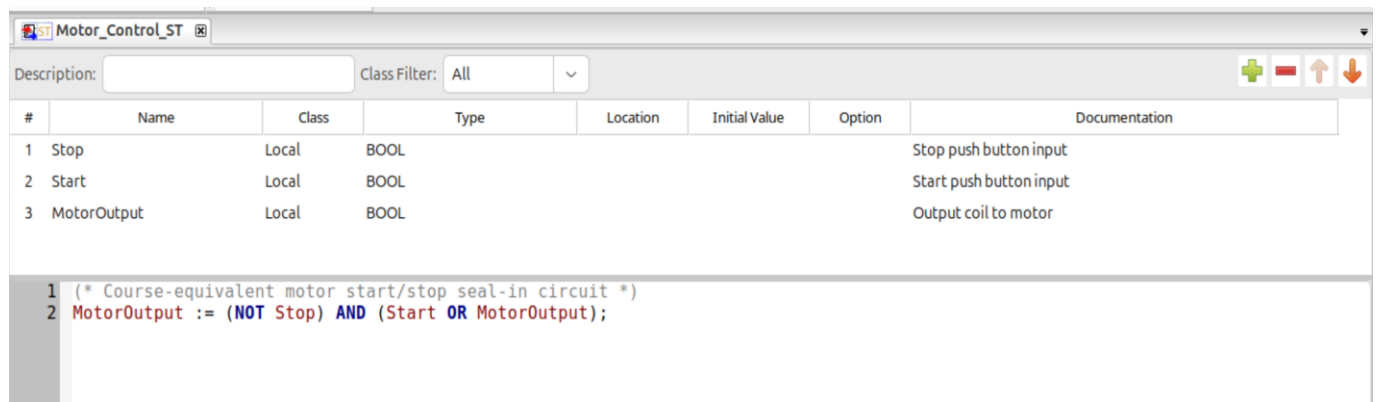
- **The Stop || tag which is normally closed is always true until input is received and becomes false**
- The Start ||/| tag which is normally open is always false until an input is received and becomes True
- The MotorOutput || which is normally open will be True when the Start tag is True and Stop is not activated, and False when the Stop tag receives an input

Step 5. Return to File > Open and load the Function Block Diagram example. Open Motor_Control_FBD and identify the block that maintains the motor output until a stop condition occurs.



The OR block maintains the motor output. Its second input (IN2) is wired to MotorOutput itself, creating a feedback loop so that once the output goes TRUE it stays TRUE until the AND block blocks it which happens when the NOT block reports that Stop has been pressed.

Step 6. Open the Structured Text example and inspect MotorControlST. Describe how the same start/stop behavior is represented using text rather than graphical contacts and coils.



The Structure text example simplified the motor control function by ensuring the MotorOuput stays True if the Stop is NOT pressed AND when either the Start is pressed or MotorOutput is already True

Step 7. Open the Instruction List example and inspect MotorControl. Record the main visual difference between this low-level textual representation and Structured Text.

Description:		Class Filter:	All	▼				
#	Name	Class	Type	Location	Initial Value	Option	Documentation	
1	Stop	Local	BOOL				Stop push button input	
2	Start	Local	BOOL				Start push button input	
3	MotorOutput	Local	BOOL				Output coil to motor	

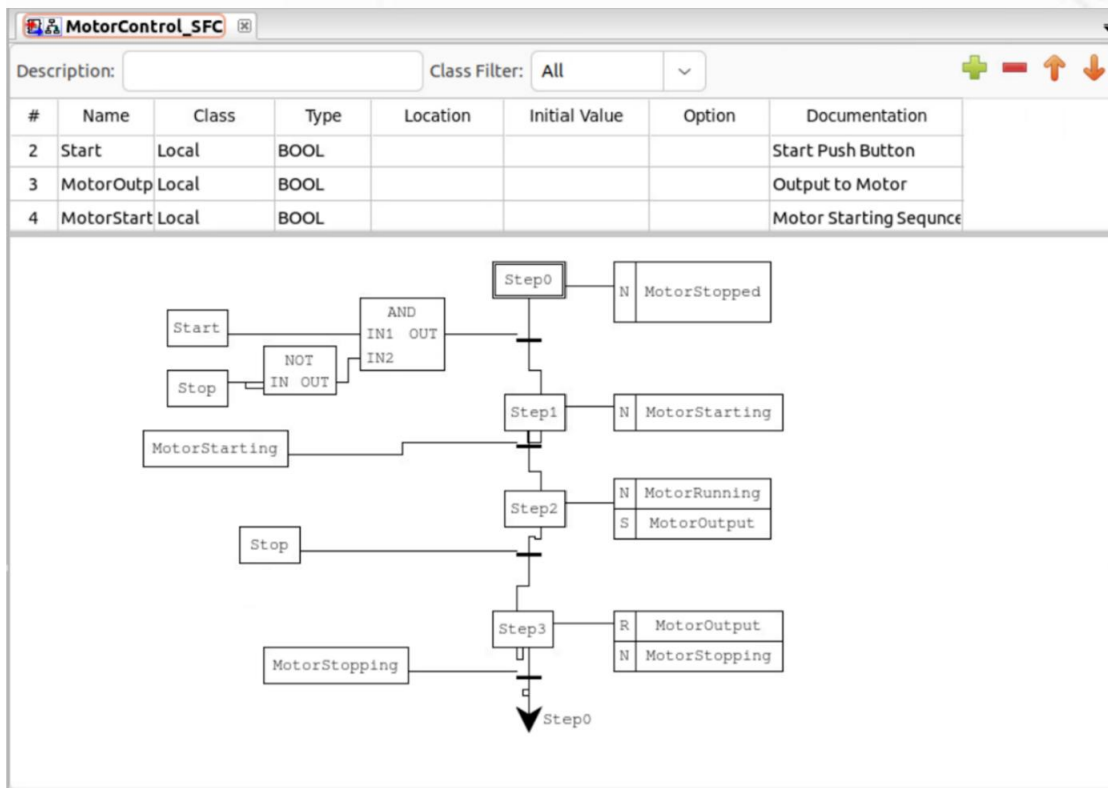
```

1 (* Legacy IEC 61131-3 Instruction List example *)
2 LD Start
3 OR MotorOutput
4 ANDN Stop
5 ST MotorOutput

```

The major difference between the Structured text and Instruction List is that Instruction List captures the step by step process of building the result in a working register, it's like a recipe book while the Structured text is one single complete equation

Step 8. Open the Sequential Function Chart example and inspect MotorControl_SFC. Identify how steps/transitions express process sequence.



Each transition has a condition that must be TRUE to move forward:

- Step0 → Step1: Start AND NOT Stop
 - Motor can start when Start is pressed and Stop is not pressed
- Step1 → Step2: MotorStarting
 - When the starting sequence completes
- Step2 → Step3: Stop

- When Stop button is pressed
4. Step3 → Step0: MotorStopping
- When stopping sequence completes, return to idle

Step 9. Complete the language comparison matrix in the lab report for all five representations.

Step 10. Capture one evidence screenshot for each programming representation, ensuring the program name is visible.

Required Evidence & Submission

- Five screenshots: Ladder Diagram, Function Block Diagram, Structured Text, Instruction List and Sequential Function Chart.
- A completed comparison matrix covering representation, strengths, limitations and a suitable industrial use case.

Feature	Strength	Limitations	Industrial use case
Ladder Diagram (LD)	Intuitive for electricians and it visually maps physical relay wirings	Becomes cluttered with complex logic	Discrete machine control, safety interlocks
Function Block Diagram (FBD)	Suitable for visualizing continuous data flow and reusable via custom blocks	Complex wire-crossing reduces readability	Process automation (water treatment, oil & gas), HVAC systems, robotics requiring signal processing
Structured Text (ST)	Suitable for algorithms, loops, arrays, strings	Harder to visualize real-time machine state without simulation tools	Complex motion control, database communication, advanced mathematical calculations
Instruction List (IL)	It has a compact code footprint	Deprecated in IEC 61131-3 (2015) due to poor readability	Legacy PLCs
Sequential Function Chart (SFC)	Suitable for mapping multi-step sequences and state machines	Managing parallel branches can become confusing if undocumented	Chemical or manufacturing processes, assembly lines

- A short explanation showing how the same motor start/stop requirement can be represented differently while preserving the same control objective.

Programming languages in PLCs are different ways of expressing the same underlying logic which gives engineers the flexibility to work in the language they are comfortable with. However, the meaning of the tags like Start, Stop, MotorOutput which are actual variables tied to the physical I/O state needs to stay consistent across all representations. This ensure predictability and prevents equipment damage, safety hazards or production downtime. Also, IEC 61131-3 was designed to ensure the same program logic could be written in any of the 5 languages without changing variable definitions

Recommended filename: C11/26/ACIS_17347_LAB02_PLC_Programming_Languages.pdf

Knowledge Check

1. Why is Ladder Diagram intuitive for technicians familiar with relay logic?

Response: Because it is a graphical based language that mimics the ladder logic used by electrical technicians when installing relay circuits.___

2. What is one advantage of Structured Text for complex calculations or conditions?

Response: _It uses the style and syntax of high-level programming languages like Python_

3. When might a Sequential Function Chart be more appropriate than a single ladder rung?

Response: _When completing recipes in batch manufacturing processes

4. Why must the meaning of tags remain consistent when the representation/language changes?

Response: _Because the language used is just the presentation layer which doesn't change how the physical processes while tags are actual variables tied to the physical I/O state. Tag consistency ensures predictability_

Marking Rubric — 100 Marks

Criterion	Marks	Standard
Technical execution	50	Lab procedure completed correctly; required configuration/analysis works as intended.
Evidence	20	Screenshots/files are complete, readable, relevant and clearly labelled.
Analysis & knowledge check	20	Responses demonstrate understanding of the industrial-control concepts observed.
Professional reporting	10	Clear structure, correct terminology, concise writing, file naming and submission compliance.
Submission Integrity: Evidence must come from the student's own assigned laboratory environment. Screenshots should show enough context to verify the activity. Do not submit another learner's screenshots or project files.		